

AccuKnox Assignment 2: AI/ML Experience, Chatbot Architecture, and Vector Databases

Zaid Shabir

January 11, 2026

1. Self-Rating and Project Overview

Self-Rating (LLM, Deep Learning, AI, ML)

Technology	Rating	Explanation
LLM	C	Limited direct experience; familiar with API usage, prompt design, and AI integration.
Deep Learning (DL)	B	Can code under supervision, with practical exposure to model pipelines.
AI	B	Can code under supervision; comfortable integrating AI components into applications.
ML	A	Can code independently and build end-to-end ML workflows.

Project Experience

OrchardIntel: Apple Disease Detector with Planet Climate Risk Advisor

- React + TypeScript application for apple leaf disease detection and farm-level risk analysis.
- Integrates Planet imagery, AOI-based visualization, climate parameters (temperature, precipitation), and dynamic farm health scoring.
- Backend managed using Supabase Edge Functions; frontend workflows enhanced using AI-assisted tools such as V0 by Vercel, Bolt New by StackBlitz, and GitHub Copilot.

Kraken – Decentralized Messaging Platform

- Secure, decentralized messaging system built on Web3 principles.
- Ethereum wallet addresses act as user identities.
- End-to-end encrypted peer-to-peer communication implemented using Gun.js.

AI-Powered Medical Diagnosis System

- Machine learning application using SVM, Logistic Regression, and Random Forest models.
- Includes preprocessing, feature scaling, training/testing pipelines, and evaluation metrics.
- Deployed via Streamlit for interactive medical prediction workflows.

Additional experiments and implementations are available on GitHub: <https://github.com/Zai14>

2. High-Level LLM Chatbot Architecture

Assumptions and Practical Context

The following assumptions are made while describing the chatbot architecture:

- Cloud-managed vector databases are acceptable for scalability and operational simplicity.
- End-to-end response latency is expected to remain under 100 milliseconds.
- The chatbot is domain-specific rather than a fully open-ended conversational agent.
- Security, reliability, and cost constraints are considered at a startup or mid-scale production level.

My understanding of these systems comes from implementing real-world projects such as *OrchardIntel* and *Kraken*. While working on these systems, I had to balance correctness, performance, and safety while integrating AI components into production-ready pipelines. This experience directly influenced the architectural decisions described below.

LLM-Based Chatbot Architecture

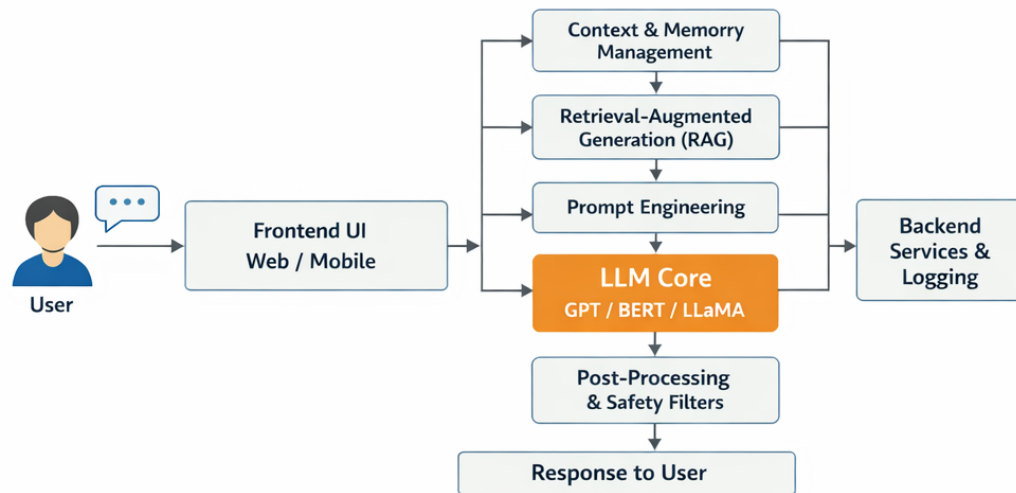


Figure 1: High-Level LLM Chatbot Architecture

Architecture Explanation

- **Frontend UI:** Web or mobile interface handling user interactions.
- **API Layer:** Manages authentication, request validation, and rate limiting.
- **Context and Memory Management:** Maintains short-term conversation history and long-term user context.
- **Prompt Engineering Layer:** Structures prompts with system instructions and contextual information.
- **Retrieval-Augmented Generation (RAG):** Retrieves relevant external knowledge from a vector database.
- **LLM Core:** Generates responses based on the constructed prompt.
- **Post-processing and Safety Filters:** Ensures safe, consistent, and formatted responses.

Flow: User Input → Frontend → API Layer → Context + RAG → Prompt Engineering → LLM → Response

3. Vector Database Overview

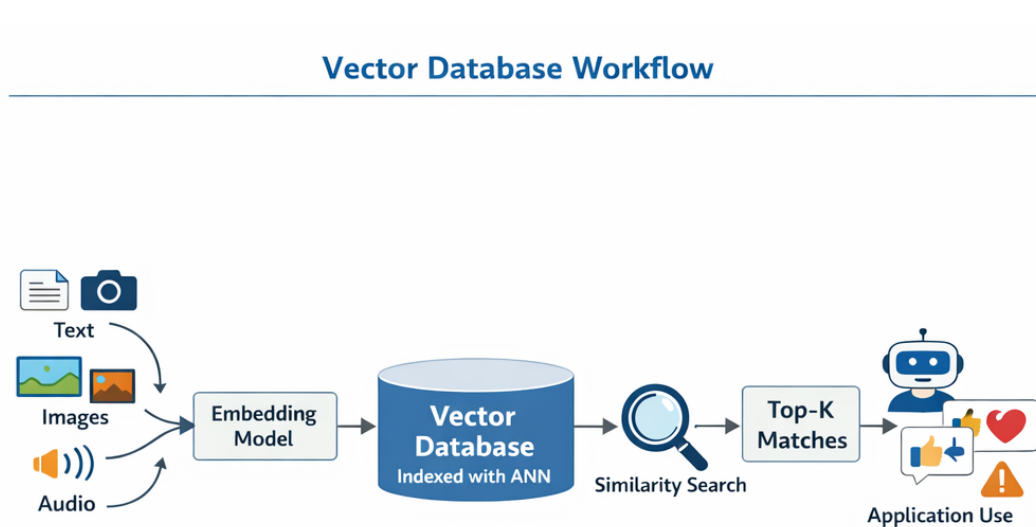


Figure 2: Vector Database Workflow

Concept Explanation

Vector databases store numerical embeddings that capture semantic meaning of unstructured data.

- Data is converted into embeddings using models such as Sentence-BERT or CLIP.
- Vectors are indexed using Approximate Nearest Neighbor algorithms.
- Queries are embedded and compared using similarity metrics.
- Retrieved vectors enhance LLM responses in RAG-based systems.

Hypothetical Use Case

A documentation chatbot serving over 10,000 technical documents with requirements of low latency, hybrid search, and high availability.

Selected Vector Database: Pinecone

- Fully managed infrastructure with high availability.
- Supports hybrid vector and metadata filtering.
- Easy integration with Python-based backends.

Alternatives considered include Weaviate, Qdrant, Milvus, Chroma, and pgvector.

4. Most Complex Python Code Written

Project: AI-Powered Medical Diagnosis System

Link: <https://github.com/zai14/AI-Powered-Medical-Diagnosis-System>

Description: Implements ML models, preprocessing pipelines, and evaluation logic using modular Python code and object-oriented design.

5. Most Complex Database Code Written

Project: Kraken – Decentralized Messaging Platform

Link: <https://github.com/zai14/Kraken-Decentralised-Messaging>

Description: Implements secure decentralized storage, authentication, and complex CRUD operations across distributed peers.

References

- GitHub: <https://github.com/Zai14>
- Pinecone Documentation: <https://www.pinecone.io/docs/>